

MT-Agent: Constructing a GUI Agent via Modality Enhancement and Text-Guided Fusion

Jinhan Dong¹, Lei Jin¹, Zhihong Zhang², Wei Tang, Runqing Zhang, Liqiang Xu,
and Junliang Xing³, *Senior Member, IEEE*

Abstract—Graphical user interfaces (GUIs) play a crucial role in facilitating user-computer interactions, making them an essential focus of research. However, current automated GUI agents face significant challenges in effectively associating task implementations with specific visual elements, and the resolution constraints of vision-language models (VLMs) also limit the richness of visual information. To this end, we propose a novel multimodal agent named MT-Agent, which enhances both textual and visual input modalities to enable the model to perceive visual elements in GUIs more effectively. Specifically, textual modality enhancement improves the semantic richness of input text by capturing task-specific details via an external VLM, while visual modality enhancement incorporates fine-grained visual details to better represent critical GUI elements. In addition, we introduce an innovative text-guided directional feature fusion mechanism, which leverages enriched text features to guide the integration with visual information. In experiments, MT-Agent demonstrated exceptional performance on AITZ dataset, achieving an action type prediction accuracy of 84.80% and a step prediction accuracy of 58.07%, surpassing previous state-of-the-art (SOTA) models. Furthermore, on the GUI Odyssey benchmark, MT-Agent achieves performance comparable to previous SOTA models while using only about 1/20 of their trainable parameters. Our codes, demos, and relevant data will be released to facilitate further research and validation within the scientific community.

Index Terms—Artificial intelligence (AI), cross-modality feature fusion, graphical user interface (GUI), modality enhancement, multimodal large language model (MLLM).

I. INTRODUCTION

GRAPHICAL user interfaces (GUIs) are fundamental to human-computer interaction, facilitating operations across diverse applications. GUI automation agents enhance

interaction efficiency by simulating user actions, handling tasks from setting alarms to executing complex workflows like product searches and software installations.

Recent progress in large language models (LLMs) has demonstrated exceptional semantic understanding and reasoning capabilities, showing great potential in handling complex tasks [1], [2], [3]. In GUI automation, LLMs support task planning and execution through their advanced language comprehension. However, early automation approaches, primarily text-based [4], [5], [6], [7], struggle with spatial reasoning, visual ambiguity, and scalability issues, highlighting the need for more robust visual-based solutions.

The rise of vision-language models (VLMs) offers a promising solution by processing image inputs directly, enabling intuitive and precise task execution. Despite progress in UI element grounding [8], [9], [10], [11] and multistep navigation [12], [13], GUI visual agents still face several challenges.

- 1) *Task Object Understanding*: GUI automation tasks require global modeling of the overall task based on instructions and a clear understanding of the task objects at the current step. However, this process often leads to confusion in models, which can negatively impact their final decisions [14], [15], [16].
- 2) *Visual Element Localization*: Accurately identifying and localizing interactive elements are critical for successful GUI automation. But low-resolution inputs often miss small icons and fine details, while redundant backgrounds in GUI screenshots add noise, diverting focus from key elements and causing errors in the specific tasks, such as navigation and interaction.

In this article, we propose MT-Agent, a vision-only GUI automation agent designed to overcome these challenges through modality enhancement and text-guided feature fusion algorithms, as illustrated in Fig. 1. Our agent employs a pure language model (LM) as the foundation, complemented by a custom-designed feature fusion algorithm to integrate visual features into the LM.

To address the ambiguity in task object understanding, we introduce textual modality enhancement (TME), which leverages VLMs as auxiliary semantic providers. Rather than generating actions directly, these VLMs offer high-level semantic insights and contextual cues, enhancing our agent's ability of global task modeling without introducing additional noise from raw predictions. For the challenge of accurate visual element localization, we design visual modality enhancement (VME) to extract fine-grained UI features and

Received 21 April 2025; revised 10 June 2025 and 31 July 2025; accepted 16 August 2025. Date of publication 19 August 2025; date of current version 24 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62472046 and Grant 82274685, and in part by the Fundamental Research Funds for the Central Universities under Grant 500422813. (Jinhan Dong and Lei Jin contributed equally to this work.) (Corresponding author: Lei Jin.)

Jinhan Dong and Lei Jin are with the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100088, China (e-mail: dongjinhan@bupt.edu.cn; jinlei@bupt.edu.cn).

Zhihong Zhang, Wei Tang, Runqing Zhang, and Liqiang Xu are with the China Mobile (Suzhou) Software Technology Company Ltd., Suzhou 215000, China (e-mail: 13581507017@139.com; reg.tang@foxmail.com; zrq1993@bupt.cn; xuxiaolling@gmail.com).

Junliang Xing is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: jlxing@tsinghua.edu.cn). Digital Object Identifier 10.1109/IJOT.2025.3600573

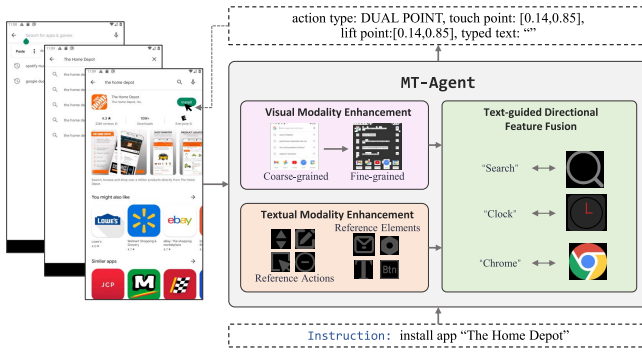


Fig. 1. MT-Agent is a purely visual GUI automation agent composed of three key components. In an episode, given one instruction, MT-Agent can provide corresponding action planning based on each screenshot.

suppress irrelevant background information. This ensures the model to concentrate on more critical and informative elements, which ensures robust performance even in complex visual scenarios.

Furthermore, we propose text-guided directional feature fusion (TDFF) to optimize multimodal integration. Unlike traditional methods, such as feature concatenation [10], [17] or gated feature fusion [18], which struggle to effectively align and prioritize the most relevant information from each modality, TDFF leverages textual information to guide the fusion process, which ensures more effective integration of multimodal features, leading to better performance. We evaluate our model on two datasets, demonstrating its strong and robust performance. Additionally, we conduct comprehensive ablation studies and module-wise analyses to assess the contribution of each component.

The main contributions are summarized as follows.

- 1) We propose MT-Agent, a GUI automation agent that enhances input information from both textual and visual modalities. Specifically, TME leverages VLMs in an innovative way to capture richer task-relevant semantics, while VME performs fine-grained encoding of input images, providing higher-quality visual features.
- 2) We propose a novel feature fusion mechanism to effectively integrate the enhanced features from the textual and visual modalities. The TDFF ensures task-specific alignment and aggregation of cross-modal features, forming the core of MT-Agent's ability to handle complex GUI automation tasks.
- 3) MT-Agent, a vision-only GUI automation model with only 350 M trainable parameters, achieves excellent performance and has been extensively tested on two datasets. It achieves state-of-the-art (SOTA) performance on AITZ, with an 84.80% accuracy in action type prediction and a 58.07% accuracy in single-step prediction, representing significant improvements of 1.91% and 4.79%, respectively, over previous SOTA models. Besides it performs comparable to that of existing leading models on GUI Odyssey, despite using significantly fewer parameters.

The remainder of this article is organized as follows. Section II reviews related work in GUI automation and

cross-modal feature integration. Section III introduces the architecture of MT-Agent, detailing the modality enhancement modules and the proposed feature fusion strategy. Section IV presents the experimental results and ablation studies conducted on benchmark datasets. Section V further analyzes TME, and discusses representative failure cases and challenges in the GUI Odyssey benchmark. Finally, Section VI concludes this article.

II. RELATED WORK

A. Agents for GUI Automation

In recent years, numerous studies have demonstrated the potential of LLMs in various domains [19], [20], [21]. As an emerging field, GUI automation tasks have also benefited significantly from these advancements. The robust semantic understanding capabilities of LLMs enable step-by-step planning, decision making, and even execution for tasks, providing a powerful tool for GUI automation [22], [23]. Furthermore, methods, such as instruction following and chain-of-thought reasoning have greatly contributed to improving task accuracy [24], [25], [26].

Early GUI agents typically relied on closed-source LLMs [27]. However, since GUI environments convey information mainly through visual elements, which LLMs struggle to process directly, additional steps were necessary, such as extracting HTML representations, accessing accessibility trees, or leveraging visual models like OCR [28] and Set-of-Mark (SoM) [5], [6], [29] to interpret interface layouts. While these approaches enabled LLMs to identify actionable elements, they faced challenges, such as dependence on external APIs, textual redundancy, and the lack of task-specific optimization, resulting in high costs, low efficiency, and limited practicality, especially when raw data access was unavailable.

To address these issues, multimodal models have been proposed [30], [31], [32], [33], [34], leveraging training to transform visual information into LLM-comprehensible features. These models bridge the gap between language and vision, excelling in screenshot-based element localization and navigation to strengthen GUI automation. However, balancing visual information with computational efficiency and optimizing visual-semantic feature fusion remain core challenges. Our work aims to develop a purely visual GUI automation agent by addressing key challenges through the introduction of advanced methods for information mining and feature fusion, thereby effectively integrating visual information into LLMs.

B. Cross-Modality Feature Processing

Cross-modality feature processing is a critical research area in multimodal learning [35], [36], [37]. Researchers have explored various methods to align and integrate visual and textual features effectively. For example, works, such as [10], [17], and [38] employ linear projection to map visual features into the same dimension as textual embeddings and concatenate them before feeding into an LM. While this approach is computationally efficient, it treats both modalities equally and lacks task-specific guidance. This often leads

to increased token length and the inclusion of irrelevant information, which may hinder downstream decoding.

To enhance alignment, BLIP-2 [39] introduces a Q -former architecture, which aligns visual and textual features in a shared embedding space through contrastive learning. However, its complexity and reliance on large-scale data limit its practicality. Meanwhile, more advanced designs such as CogVLM [40] incorporate visual expert modules and hierarchical interaction layers to achieve deep multilevel fusion. While these models exhibit strong performance, they significantly increase model complexity and training cost.

Unlike approaches that introduce new decoding tokens through feature concatenation, Auto-GUI [18] employs a gated fusion mechanism that integrates visual features directly into textual representations. This method reduces computational overhead by avoiding token expansion and enables dynamic weighting between modalities. However, its fusion strategy applies gating globally and uniformly, lacking the granularity to model localized cross-modal interactions. Moreover, the semantic gap between modalities remains insufficiently addressed, which may hinder performance in fine-grained, context-sensitive tasks.

To address these limitations, we propose a novel feature fusion mechanism that leverages a text-guided, directional attention strategy, named TDFF. Our method extracts task-relevant semantics from language input and uses this information to selectively attend to visual features, enabling interpretable, fine-grained fusion. This design achieves a better balance between performance and efficiency, offering a compact yet expressive cross-modal representation.

III. METHOD

In this section, we first present an overview of the overall architecture of our model. Following this, Section III-B delves into the TME module, Section III-C focuses on the VME module, and Section III-D explores the TDFF module. Finally, we provide a detailed description of our training and inference processes in Section III-E.

A. Overview

1) *Task Description*: At the step t , given a GUI screenshot I_t with a text T_t that contains user instruction T^i (a task has only one instruction) and historical operations T_t^h , the GUI automation task aims to predict the current action based on two parts of the above information to advance the process until the task is completed.

2) *Overall Architecture*: As illustrated in Fig. 2, our model separately conducts modality enhancement of textual and visual information and then fuses them with a fusion module. Specifically, for the two parts of input information, TME employs a VLM as an assistant to comprehensively understand the screenshot I_t and the instruction T^i in the text information, providing referential suggestions. These suggestions, along with the entire text content T_t , are encoded into a feature F_{tme} through a language encoder. Meanwhile, VME utilizes both a fine-grained and a coarse-grained vision encoder, and

enhances the coarse-grained image features with the fine-grained ones to obtain F_{vme} . Finally, these two parts of features are effectively fused through TDFF and decoded into text output by a language decoder. This output represents the model's prediction of the current action, and it will be added to the historical operation T_{t+1}^h as the input text T_{t+1} for the next step.

B. Textual Modality Enhancement

According to an individual task instruction, models often struggle to model the connection between each screenshot and current action, leading to a low accuracy rate in action prediction. To mitigate this issue, recent studies have started leveraging powerful VLMs as external planners to assist with understanding and decision making [9], [41], [42]. However, since the generation of planning information depends on the historical actions, it cannot be produced in a single forward pass, which limits its applicability as direct training data in supervised learning pipelines.

To address this limitation, we adopt a new approach in which the VLM serves as an assistant rather than a decision maker, and its outputs are utilized as supplementary references to augment the originally simple task instructions without being directly involved in model inference. Unlike previous works, we remove historical context from the model's input and ensure that its responses are based solely on the current instruction and screenshot. This design simplifies the reasoning process, reduces computational cost, and avoids potential bias from past interactions. In this setting, there are two relatively simple tasks that assistant needs to focus on: 1) Based on the instruction and the current image, determine which action type (e.g., stop, click, type, and scroll) is most effective in advancing the target and bringing it closer to completing the instruction and 2) If a click operation is currently needed, which element is the most likely to be clicked. The generated responses will be added to the text input, transforming the original single and hard-to-understand task instructions into more informative and clearly defined text information.

Prompt: We present the prompt we used in Fig. 3. To ensure that the output of the assistant meets the format requirements and reduces the probability of its "free play," we have set several options for questions for the model to choose from, similar to the method in [4]. These options will constrain the responses generated by the model, which ensures that the assistant's responses adhere to established format while also enabling it to provide more comprehensive information.

C. Visual Modality Enhancement

Early VLMs often compress input images to a resolution of 224×224 pixels [39], [43], [44], which can lead to distortion or deformation of small icons in GUI interfaces, thereby compromising the model's ability in grounding. Therefore, we propose a method that can obtain more fine-grained image features, as shown in the upper left part of Fig. 2.

1) *Cropping and Fine-Grained Encoding*: It is evident that the aspect ratio of GUI interface screenshots typically is approximately 2:1 (e.g., 540×1170 , 720×1440).

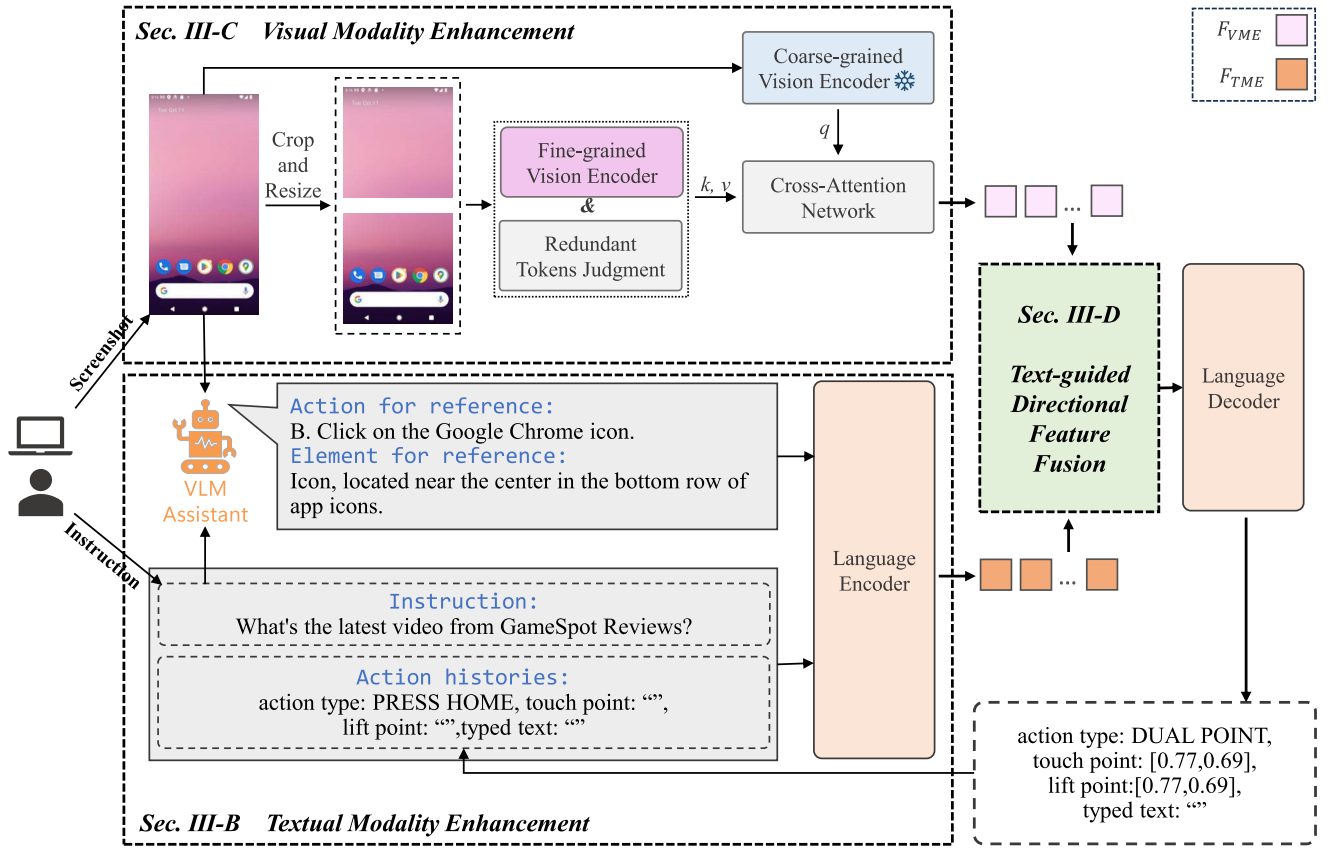


Fig. 2. Overview of MT-Agent. In this figure, the lower left corner represents the TME module, detailed in Section III-B, while the upper left corner illustrates the VME module, detailed in Section III-C, and the right part depicts our proposed TDFF module, detailed in Section III-D.

SYSTEM#

You are a user operating a graphical user interface. You will receive an instruction and a screenshot. You need to determine what action (task completed, click, type, scroll) the current interface should perform to ultimately complete the instruction, and provide the most interactive element when it is determined to be a click.

USER#

Task Description

Read the task requirements *<instruction>*. Evaluate whether the task is complete based on the provided screenshot. If the task is incomplete, identify the next step (click, type, scroll) required to progress toward completing the task. For click actions, specify the element's type (icon or text) and its relative position.

Action Planning and Element Selection

If the task is complete: Confirm "Task completed."

If further action is needed: Specify the next action and identify the relevant interactive element.

Output Format:

Action: Use the following options as a template for the format but do not copy the options exactly. Choose the most relevant option and fill in details:

A. Task completed B. Click C. Type text D. Scroll (specify direction: up, down, left, or right)

Element Information: If "Click" is selected, describe the element's type (icon, text) and its relative position (e.g., top-right, center, bottom-left).

Fig. 3. Template of prompt. The *<instruction>* in the task description section will be filled with specific task instruction. This template specifies actionable options, effectively constraining the output space of the VLM. By doing so, it ensures that the generated responses are more accurate, standardized, and aligned with the intended requirements. Moreover, the template requires VLM to provide detailed descriptions of interactive elements for click actions, including their type and relative position, thereby offering richer and more detailed information.

Consequently, prior to encoding, We crop the image into two segments along the longer side and encode each segment using the same visual encoder, which accepts 448×448 pixel images, to extract fine-grained image features. This method not only ensures that the aspect ratios of the images remain largely consistent but also approximately halves the computational cost. For instance, consider an image divided into $2H$ rows and

W columns of patches, without cropping, the computational complexity of self-attention is $O_{w/o} = (2WH)^2 = 4(WH)^2$. In contrast, our cropping method reduces this complexity to $O_w = 2(WH)^2$, effectively cutting the computation in half.

2) *Redundant Tokens Judgment*: Redundant tokens are prevalent in the image features of GUIs. This is due to the fact that GUIs typically comprise multiple elements, concentrating

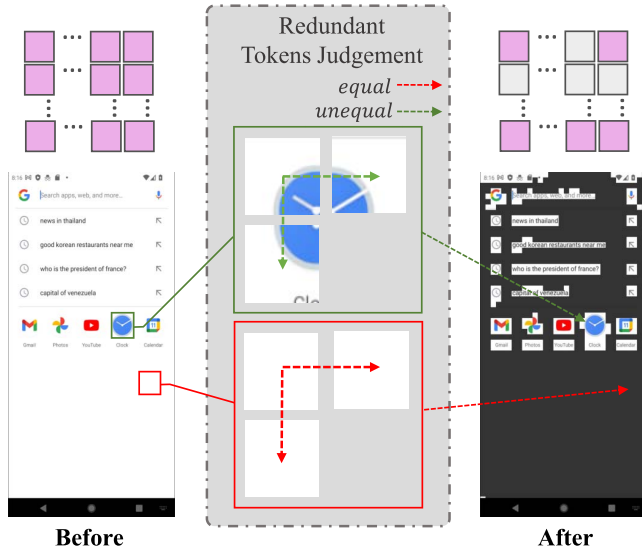


Fig. 4. Example of redundant tokens judgment. The left side of the figure displays the original screenshot along with its corresponding tokens. During the judgment process, each patch in the “alarm clock” icon is inconsistent with its adjacent patches, thus being determined as valid patches. However, the white background part is judged as redundant, because each patch is exactly the same as the nearby patches. The right side illustrates the screenshot and tokens following the redundant tokens judgment. The blackened parts in the screenshot indicate that the patches here are determined to be redundant, corresponding to the grayed parts in the tokens.

effective tokens within these components. Conversely, the background often consists of numerous repetitive pixels, resulting in tokens that carry significantly less information. Such redundant information introduces considerable noise to the model. Therefore, we design a simple yet efficient judgment mechanism to mitigate this impact. Specifically, we create a list with a length equal to the number of patches (which is also equal to the number of image tokens), initializing each element to 1, which means that the initial weight of each patch is 1. Subsequently, we iterate through each patch in the image. If the pixel value of a given patch is nearly identical to that of the adjacent patch to its right or below, we determine that the current patch is redundant and reduce the weight of the corresponding token by multiplying it with a parameter W_r , which is elaborated in Section IV-D. Fig. 4 presents an example of the proposed judgment mechanism. There are a large number of redundant patches in the image, We identify them and reduce the weights of the corresponding tokens.

Then, we enhance the original coarse-grained image features with the fine-grained features to obtain F_{vme} , which integrates global context from the screenshot with enriched fine-grained details, providing a more comprehensive visual representation. The overall process is as follows:

$$F_{image}^{coarse} = \text{CoarseEncoder}(I_t) \quad (1)$$

$$F_{image}^{fine} = J(\text{FineEncoder}(I_t)) \quad (2)$$

$$F_{enhanced} = \text{CrossAtten}(F_{image}^{coarse}, F_{image}^{fine}) \quad (3)$$

$$F_{vme} = F_{image}^{coarse} + W * F_{enhanced} \quad (4)$$

where I_t is the input screenshot at step t , *CoarseEncoder*, *FineEncoder* indicates the coarse-grained vision encoder and

the fine-grained vision encoder, respectively. J means the redundant tokens judgment. $F_{enhanced}$ represents the attention information from the coarse-grained features to the fine-grained features, and W is a trainable parameter, initialized by W_e which enables the model to autonomously learn which aspects of the attention information should be emphasized during feature enhancement.

D. Text-Guided Directional Feature Fusion

After obtaining the modality-enhanced text features F_{tme} and image features F_{vme} , we propose a directional fusion strategy to integrate the two kind of features, as illustrated in Fig. 5. Compared to previous approaches [10], [17], [45], direct concatenation of features, while straightforward, not only increases the number of tokens that require decoding but also introduces a substantial amount of irrelevant information into downstream tasks, making it difficult for the model to distinguish key features. Although gate-based fusion [18] introduces a weighting mechanism, its global weighting nature limits its capability to capture fine-grained interactions between modalities.

Our directional fusion method addresses these limitations by introducing a set of learnable queries, which perform cross-attention with both text and image features. These queries act as intermediaries, guiding the text and image features into a unified representation space. They first engage with the text features to extract critical textual information, such as task instructions and notable elements. Subsequently, guided by this textual information, they interact with the image features by focusing on its details, such as identifying evidence of task completion or locating interactive elements that can advance the task. This process results in a directional fusion of information from both textual and visual modalities. The detailed process is as follows:

$$F_{tme} = \text{TME}(T_t, I_t) \quad (5)$$

$$F_{vme} = \text{VME}(I_t) \quad (6)$$

$$Q_{text} = \text{CrossAtten}(Q_{init}, F_{tme}) \quad (7)$$

$$F_{fused} = \text{CrossAtten}(Q_{text}, F_{vme}) \quad (8)$$

where T_t and I_t represent the input text and screenshot at the current step, respectively, while Q_{init} denotes a set of learnable queries. F_{tme} and F_{vme} are TME and VME features, which are introduced above. *CrossAtten* refers to the cross-attention network, and Q_{text} is the queries enriched with task-relevant textual information, derived through the cross-attention mechanism between Q_{init} and F_{tme} . Through the use of these queries, the cross-attention mechanism achieves point-wise matching between text and image features and obtains aggregated features F_{fused} . This approach preserves the deep semantic relationships between modalities more effectively than simple concatenation or gating operations, guided by the text features. Ultimately, we obtain the hidden layer features as follows:

$$F_{concat} = \text{Concat}(F_{tme}, F_{fused}) \quad (9)$$

$$H_{hidden} = F_{tme} + \text{SLP}(F_{concat}) \quad (10)$$

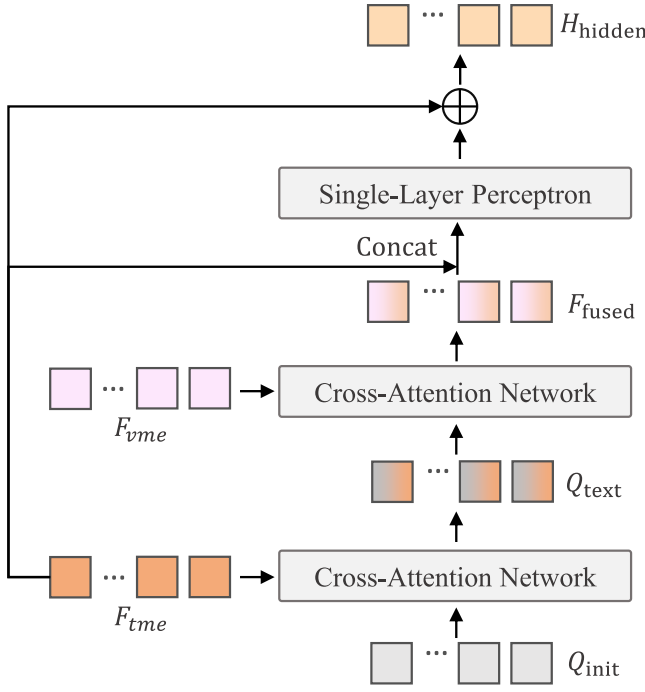


Fig. 5. Framework of the TDFF module.

TABLE I
DETAILED STATISTICS OF THE AITZ DATASET

Subset	Train		Test	
	Episodes	Screens	Episodes	Screens
GENERAL	323	2405	156	1202
INSTALL	286	2519	134	1108
GOOGLEAPPS	166	1268	76	621
SINGLE	844	2594	0	0
WEBSHOPPING	379	5133	140	1793
Total	1998	13919	506	4724

where F_{concat} refers to the features obtained by concatenating the features of F_{tme} and F_{fused} , and SLP refers to the Single-Layer Perceptron. Finally, H_{hidden} will be decoded into the final textual output using an language decoder.

This feature fusion method we employ neither increases the number of tokens requiring decoding nor compromises the integrity of the original textual information. Instead, it selectively integrates critical visual information in a directed manner, significantly enhancing the model's focus on relevant information and improving its overall performance. In addition, it is thoroughly validated in Section IV-C.

E. Training and Inference Strategy

1) *Trainable Components*: Our model is composed of multiple modules, some of which are based on pretrained models and others are newly trained. Specifically, the foundational LM is initialized from Flan-Alpaca [46] and further fine-tuned in our training process, including both its language encoder and language decoder. The TME module utilizes VLM as a frozen assistant, and the coarse-grained visual encoder in

the VME module is a frozen BLIP2 [39] image encoder. In contrast, the fine-grained visual encoder is built upon the EVA-CLIP [43] architecture and is trained from scratch. The feature fusion module and all cross-modal interaction layers are also newly introduced and trained. In total, approximately 350 M parameters participate in training.

2) *Training Objective*: We employ the cross-entropy loss L_{ce} as the supervision objective, which guides the model to learn semantically grounded and coherent action sequences. The loss is computed between the predicted token sequence Y_{pred} and the ground truth sequence Y_{target}

$$L_{ce} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T y_{i,t} \log \hat{y}_{i,t} \quad (11)$$

where N represents the batch size, T represents the sequence length, $y_{i,t}$ is the true label of the i th sample at time step t , and $\hat{y}_{i,t}$ is the predicted probability of the model at time step t for the i th sample.

3) *Inference*: During inference, we follow an auto-regressive decoding strategy. Starting from a special [BOS] token, the model generates tokens one by one, conditioned on previous outputs. Meanwhile, we structure the input instructions and screenshots consistently with the training format to ensure more stable and accurate model outputs.

IV. EXPERIMENTS

A. Experiment Setup

1) *Dataset*: We use AITZ [42] as our main dataset, which is a refined version of AITW [22]. The dataset comprises 1998 training segments with a total of 13 919 images, and 506 test segments with 4724 images, with a training-to-test split ratio of 7:3. Table I illustrates the statistics of specific data. Notably, we only utilize the screenshot and instruction pairs from the dataset but do not use the Chain-of-Action-Thought (CoAT) portion.

Furthermore, we conducted experiments on the GUI Odyssey [13] to provide a more comprehensive evaluation of our method. The GUI Odyssey comprises 7735 cross-application task scenarios. Following the random splitting method described in the article, we partitioned the dataset into training and test sets at a ratio of 3:1, resulting in 5802 training samples and 1933 test samples. Additionally, the test set was categorized into six distinct types based on task categories: 1) general tools; 2) information management; 3) Web shopping; 4) media entertainment; 5) social sharing; and 6) multiapp tasks, as detailed in Table II.

2) *Metrics*: For AITZ, following [18], we adopt the screen-wise action matching score (AMS) to evaluate the success rate of actions in automated GUI tasks. If the action type and double tap gesture of a screenshot match the gold gestures, the action prediction for the current screenshot is considered correct. Specifically, type accuracy (type_acc) reflects whether the predicted action type is correct, while step accuracy (step_acc) requires both the correct action type and its associated value to match the ground truth. Additionally, similar to the approach of [42], we introduce additional metrics, such as click accuracy (click_acc), text input accuracy (text_acc),

TABLE II
DETAILED STATISTICS OF THE GUI ODYSSEY DATASET

Subset	Train		Test	
	Episodes	Screens	Episodes	Screens
TOOL	1358	16835	463	5665
INFORMATION	1102	18086	321	5357
SHOPPING	360	6807	119	2241
MEDIA	792	8224	288	3012
SOCIAL	967	13346	349	4752
MULTIAPPS	1223	26067	393	8399
Total	5802	89365	1933	29426

and task completion accuracy (stop_acc), to provide a more comprehensive assessment of the model's performance.

For GUI Odyssey, we adopt the identical metric utilized in the article [13], specifically the AMS. This score indicates that an action is deemed correct if both its type and value are accurately predicted, which is equivalent to the previously mentioned step_acc.

3) *Implementation Details*: During training process, we set the number of training epochs to 20, the batch size to 4, the learning rate to 1×10^{-4} , and the warmup_ratio to 0.05. Other initialization settings were adopted from [18]. The model training was carried out on 4 NVIDIA 3090 GPUs and 24-GB memory per card.

B. Main Result

1) *AITZ*: Table III presents the primary experimental results on AITZ [42], highlighting that MT-Agent outperforms all existing models on the AITZ dataset and achieves SOTA performance. We trained Flan-Alpaca as a baseline for comparison with MT-Agent, for the visual feature input, we employed a feature concatenation approach, where the image features are projected to the same dimension as the text features through a linear layer and then concatenated together.

MT-Agent demonstrates significant performance improvements over CogAgent [32] and Auto-GUI [18] in almost all metrics. Even when compared with models employing the CoAT paradigm, MT-Agent achieves superior overall performance, and it notably surpasses the Auto-GUI+CoAT [42] and CogAgent+CoAT [42] in both action type prediction accuracy and single-step prediction accuracy. Furthermore, while Auto-GUI requires 220 million training parameters, MT-Agent only adds an additional 130 million trainable parameters, totaling 350 million, yet achieves a substantial improvement. In particular, MT-Agent's click_acc exceeds 50%, more than half higher than Auto-GUI+CoAT's 32.20%.

Moreover, VMT-Agent is a variant of our model that excludes the text modality enhancement (TME) component. It demonstrates that even without the guidance of external information, our approach still shows remarkable performance. It not only surpass Auto-GUI with external information guidance (+CoAT) but also slightly outperforms CogAgent (+CoAT) which has the external guidance.

2) *GUI Odyssey*: The experimental results on GUI Odyssey [13] are shown in Table IV. Compared to the baseline

model, our model has achieved a substantial improvement in overall accuracy by 13.08%, which further validates the effectiveness of our proposed method. Notably, the performance improvements in the media and social categories are particularly significant, with accuracy increases of 16.93% and 14.94%, respectively. These results indicate that our model demonstrates superior adaptability in tasks closely related to real-life scenarios. Furthermore, despite having nearly twenty times fewer training parameters compared to larger models [8], [13], [47], [48], our model achieves comparable or even superior performance. This finding underscores that our model can maintain high performance levels even with a substantial reduction in parameter count, thereby highlighting the efficiency and competitiveness of the proposed approach.

C. Ablation Study

To comprehensively evaluate the contribution of individual components and their constituent elements to the overall performance of MT-Agent, we performed systematic ablation studies on the AITZ dataset. In addition to the step_acc rate and the action type_acc rate, we have also included the metric of click_acc to provide a more comprehensive analysis of the role of each component.

1) *Analysis of the TME Module*: To evaluate the effectiveness and robustness of the TME module, we designed two sets of experiments using different assistant models: 1) GPT-4o [27] and 2) the smaller open-source Qwen2-VL-7B [49]. As shown in Table V, TME consistently improves model performance across both settings. With GPT-4o as the assistant, integrating TME boosts the step_acc of Flan-Alpaca* from 43.90% to 48.41% and MT-Agent from 53.41% to 58.07%. By providing additional auxiliary information related to the current task, including possible actions to execute and elements to interact, the TME module effectively guides the model toward more accurate predictions.

To further validate the robustness of our approach, we replaced GPT-4o with Qwen2-VL-7B. Despite reduced model capacity, TME still yields notable improvements: step_acc increases from 43.90% to 45.51% for Flan-Alpaca* and from 53.41% to 55.46% for MT-Agent. These results demonstrate that TME is not only effective in enhancing decision-making but also adaptable to assistants with varying capabilities, confirming its robustness and broad applicability.

2) *Analysis of the Action Histories*: To assess the effectiveness of historical contextual information, we conduct a set of experiments to quantify the impact of historical actions on model performance. In this context, action histories refer to the sequence of previously executed actions, which are provided to the model as additional input to help it understand the ongoing task trajectory. As shown in Table V, incorporating more historical actions consistently improves performance up to a history length of 8, where the model achieves the highest accuracy of 58.07%. In contrast, removing all historical information leads to a notable drop to 53.13%. These results

TABLE III

PERFORMANCE ON AITZ. “TYPE_ACC” IS THE ACCURACY FOR ACTION TYPE PREDICTION, AND “STEP_ACC” INDICATES SINGLE-STEP PREDICTION ACCURACY. “*” MEANS THAT FLAN-ALPACA AS A PURE LM SHOULD BE INCORPORATED WITH AN ADDITIONAL FEATURE FUSION MODULE TO ENABLE ITS VISUAL FUNCTIONALITY, SO WE ADDED A FEATURE CONCATENATION LAYER TO ITS ARCHITECTURE. VMT-AGENT IS A VARIANT WITHOUT TME. THE TRAINABLE PARAMETERS MARKED WITH A “-” SYMBOL INDICATES THAT THE MODEL IS ZERO-SHOT IN THE EXPERIMENT. APART FROM THE TRAINABLE PARAMETERS, THERE ARE ADDITIONAL COMPONENTS THAT REQUIRE FURTHER EXPLANATION. AUTO-GUI [18] INCORPORATES A FROZEN BLIP2 [39] IMAGE ENCODER. COGAGENT [32] IS A PRETRAINED MODEL WITH 18 B PARAMETERS. “CoAT” [42] REFERS TO THE INTEGRATION OF SUPPLEMENTARY INFORMATION PROVIDED BY GPT-4V. FLAN-ALPACA [46] AND VMT-AGENT (OURS) BOTH INCLUDE A FROZEN BLIP2 [39] IMAGE ENCODER. MT-AGENT (OURS) CONTAINS A FROZEN BLIP2 [39] IMAGE ENCODER AND LEVERAGES ADDITIONAL INFORMATION SUPPLIED BY GPT-4O

Models	Trainable parameters	Accuracy for detail			Type_acc	Step_acc
		Click_acc	Text_acc	Stop_acc		
AUTO-GUI [18]	220M	12.72	67.80	60.12	73.79	34.46
+CoAT [42]		32.20	81.40	74.40	82.89	47.49
CogAgent [32]	-	51.50	34.00	4.76	65.86	44.52
+CoAT [42]		66.15	21.80	24.60	72.59	53.28
Flan-Alpaca* [46]	250M	41.20	76.20	27.98	79.57	43.90
VMT-Agent(Ours)	350M	46.09	73.60	69.05	82.28	53.41
MT-Agent(Ours)	350M	51.24	78.00	75.00	84.80	58.07

TABLE IV

PERFORMANCE ON GUI ODYSSEY. “†” INDICATES THAT FOR EACH CATEGORY, THIS MODEL SELECTED ONLY 500 SAMPLES FOR TESTING. THE TRAINABLE PARAMETERS MARKED WITH A “-” SYMBOL INDICATES THAT THE METHOD INVOKES THE API FOR EXPERIMENTATION. VMT-AGENT IS A VARIANT WITHOUT TME. THE GRAY-MARKED LINES INDICATE THAT THE TRAINABLE PARAMETERS OF THIS METHOD ARE SIGNIFICANTLY LARGER THAN OUR MODEL. APART FROM THE TRAINABLE PARAMETERS, THERE ARE ADDITIONAL COMPONENTS THAT REQUIRE FURTHER EXPLANATION. AUTO-GUI [18] INCORPORATES A FROZEN BLIP2 [39] IMAGE ENCODER. FLAN-ALPACA* [46] AND VMT-AGENT (OURS) BOTH INCLUDE A FROZEN BLIP2 [39] IMAGE ENCODER. MT-AGENT (OURS) CONTAINS A FROZEN BLIP2 [39] IMAGE ENCODER AND LEVERAGES ADDITIONAL INFORMATION SUPPLIED BY GPT-4O

Models	Trainable parameters	Tool	Information	Shopping	Media	Social	Multi-Apps	Overall
GPT-4V† [27]	-	23.49	20.16	19.15	16.92	13.83	19.02	18.76
GPT-4o† [27]	-	20.81	16.28	31.91	15.38	21.28	16.67	20.39
Auto-GUI [18]	220M	73.33	55.65	53.46	66.77	64.75	67.10	63.51
SeeClick [8]	7B	-	-	-	-	-	-	53.92
OS-Atlas-7B [47]	7B	-	-	-	-	-	-	61.98
Qwen-VL [48]	7B	83.11	65.70	62.43	76.38	76.12	73.10	72.81
OdysseyAgent [13]	7B	85.16	68.53	62.87	76.49	77.61	74.83	74.25
Flan-Alpaca* [46]	250M	70.01	51.47	52.34	60.56	60.69	62.79	59.64
VMT-Agent(Ours)	350M	79.03	60.26	57.74	73.24	73.15	72.23	69.28
MT-Agent(Ours)	350M	81.99	65.15	61.54	77.49	75.63	74.49	72.72

highlight the importance of historical context for accurate multistep decision-making.

3) *Analysis of the VME Module*: In this part, we systematically removed one component at a step to validate the impact of each part of the VME module, and thoroughly analyze the role of them from multiple perspectives. Specifically, in the “Component” column of Table V, “enhancement” represents using fine-grained image features to enhance coarse-grained image features, eliminating this parts means that during the feature fusion stage, the text features will interact separately with these two kinds of image features to obtain the fused representation.

In this study we observed that the “Type_acc” differences among the various model variants were relatively minor, suggesting that the text understanding capability of the model remained largely unaffected across these variants. This observation may be attributed to that the VME module having a limited influence on model’s text comprehension. However,

after incorporating VME, the overall accuracy of the model increased by 3.10% compared to the baseline model without it, with a particularly significant improvement in “Click_acc,” which increased by 4.29%. This result clearly demonstrates the effectiveness of the VME module in enhancing the model’s ability to interpret user interaction behaviors.

Additionally, we conducted a detailed analysis of the importance of each submodule within the VME module. Although the cropping strategy resulted in a modest increase in model accuracy, it significantly improved the training speed from 4.8 to 5.8 samples per second, representing a 20.83% increase. And the experiments revealed that reducing the redundant token judgment or feature enhancement components led to a general decrease in model accuracy by 5.74% and 3.56%, respectively. This decline was primarily due to reduced click_acc, indicating that these submodules play a critical role in capturing key visual information and optimizing feature representation.

TABLE V

ABLATION EXPERIMENTS (A) ABLATION STUDIES OF THE TME MODULE (B) IMPACT OF ACTION HISTORIES ON MODEL PERFORMANCE (C) ABLATION STUDIES OF THE VME MODULE. “-” DENOTES THE REMOVAL OF THIS PART DURING THE EXPERIMENT, WHILE “N” INDICATES THAT NO COMPONENTS HAVE BEEN REMOVED. “CROPPING” REFERS TO THE CROPPING STRATEGY, “REDUNDANT” DENOTES THE REDUNDANT TOKENS JUDGMENT, AND “ENHANCEMENT” REPRESENTS THE FEATURES ENHANCEMENT METHOD (D) IMPACT OF DIFFERENT FEATURE FUSION STRATEGIES ON PERFORMANCE. “CONCAT” REPRESENTS THE METHOD OF DIRECTLY CONCATENATING VISUAL FEATURES WITH TEXT FEATURES. “GATED FUSION” INDICATES THE GATED FUSION METHOD

(a)

Models	TME	Assistant	Click_acc	Type_acc	Step_acc
Flan-Alpaca*	×	/	41.20	79.75	43.90
	✓	Qwen2-VL	41.35	79.66	45.51
	✓	GPT-4o	42.31	79.95	48.41
MT-Agent	×	/	46.90	82.28	53.41
	✓	Qwen2-VL	48.01	84.08	55.46
	✓	GPT-4o	51.24	84.80	58.07

(b)

Action histories	Click_acc	Type_acc	Step_acc
0	48.61	82.05	53.13
2	50.40	84.27	56.29
4	50.40	84.70	57.13
6	50.49	85.31	57.75
8	51.24	84.80	58.07
10	49.96	84.61	56.22

(c)

VME	Component	Click_acc	Type_acc	Step_acc
×	N	46.95	84.76	54.97
✓	- Cropping	50.46	85.25	57.35
	- Redundant	43.36	85.41	52.33
	- Enhancement	47.05	85.50	54.51
	N	51.24	84.80	58.07

(d)

Strategy	Click_acc	Type_acc	Step_acc
Concat	41.98	82.35	50.00
Gated Fusion	42.10	85.25	53.17
TDFF(ours)	51.24	84.80	58.07

TABLE VI

IMPACT OF TOKEN WEIGHTS ASSOCIATED WITH REDUNDANT PATCHES

W_r	Click_acc	Type_acc	Step_acc
1×10^{-3}	50.19	85.25	56.96
1×10^{-4}	50.28	85.16	57.11
1×10^{-5}	51.24	84.84	58.07
1×10^{-6}	48.79	85.06	56.58
1×10^{-7}	49.21	85.01	56.16

4) *Analysis of the TDFF Module*: The results are presented in Table V. Notably, the performance of feature concatenation is particularly suboptimal, likely due to the substantial

TABLE VII

IMPACT OF COARSE-GRAINED IMAGE FEATURES ON THE ATTENTION TO FINE-GRAINED FEATURES

W_e	Click_acc	Type_acc	Step_acc
1×10^{-3}	50.03	85.14	57.73
1×10^{-4}	49.06	84.95	56.86
1×10^{-5}	51.24	84.84	58.07
1×10^{-6}	49.72	85.35	57.03
1×10^{-7}	49.00	84.70	56.69

semantic discrepancy between text and image features. Despite both types of features undergoing modality enhancement and containing richer information, the concatenated features fail to effectively align these two modalities. Consequently, the decoder of our LM struggles to process this part of the features adequately. Although the gated fusion method achieved a 3.17% improvement in accuracy compared to the feature concatenation method, it still exhibited a 4.90% gap relative to our proposed TDFF method, thereby validating the superior effectiveness of our approach.

D. Sensitive Experiments

In this part, we conducted more experiments to deeply analyze the influence of some key parameters in the MT-Agent on the model performance. All experiments followed the original settings, and only one hyperparameter was adjusted each time to compare its impact on the model performance.

We first examined the impact of the parameter W_r , which is used to adjust the weight of redundant patches as described in Section III-C, on the model’s performance. Specifically, this parameter represents the weight given to tokens associated with redundant patches, therefore, a smaller value of W_r indicates that the model allocates less attention to these regions. As shown in Table VI, both excessively large and excessively small values of this parameter can lead to a reduction in accuracy. A large weight for redundant tokens introduces additional noise, whereas a small weight results in a substantial loss of global information. As shown in the table, The model’s performance is optimized when this parameter is set to 1×10^{-5} .

Subsequently, we examined the influence of the parameter W , as presented in (4), on the model’s performance. This parameter, initialized by W_e , controls the extent of attention to fine-grained image features within the VME module. In our approach, coarse-grained image features capture the global context of the image, while fine-grained features provide more detailed and specific local information. Consequently, a higher value of W_e indicates that the model places more emphasis on fine-grained image features, whereas a lower value suggests a stronger focus on global information. This sensitivity experiment aims to determine the optimal initial value of W_e to achieve a balanced integration of these two types of features, thereby maximizing model performance. As illustrated in Table VII, setting W_e to 1×10^{-5} results in

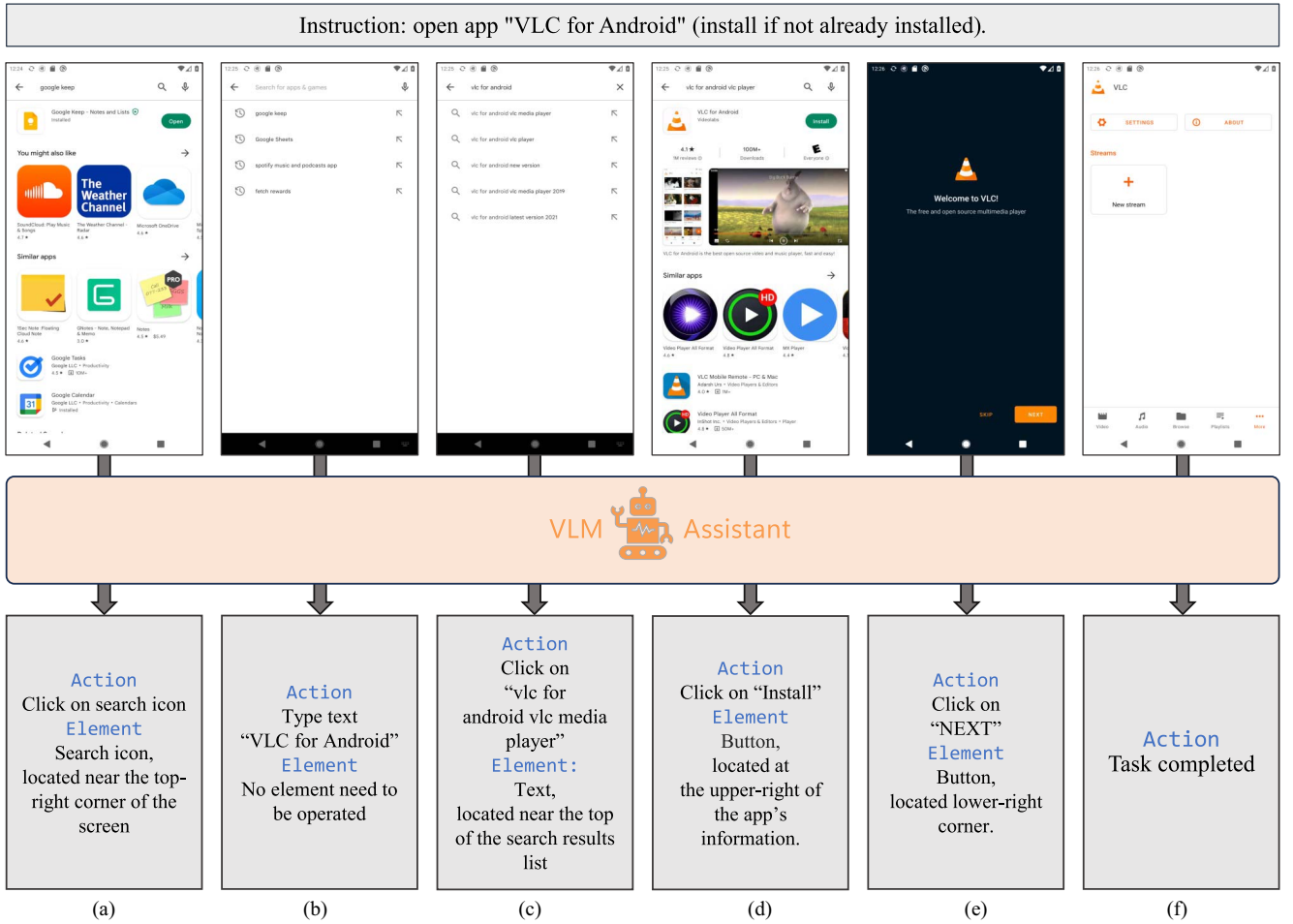


Fig. 6. Reply from the VLM assistant. The figure illustrates the responses generated by the assistant for each step of the task within a given episode. The "instruction" refers to the objective of the current task. The upper portion of the image displays the input screenshot, while the lower portion shows the corresponding response produced by the assistant.

peak model performance, indicating an ideal balance between coarse-grained and fine-grained features.

E. More Experiment Results

1) *Responses From the VLM Assistant in TME:* As shown in Fig. 6, the assistant begins by making an initial, high-level judgment regarding the nature of the current task. If the task does not involve a click action, as in cases (b) or (f), the assistant abstains from making further predictions about interactive elements, since these actions do not necessitate additional element-specific interactions. On the other hand, in cases (a), (c), (d), and (e), when the assistant predicts that the current action involves a click, it dynamically identifies and highlights the most relevant element for interaction. For these click-type actions, the assistant not only determines the type of the element but also its relative position within the user interface, thereby providing more comprehensive information.

2) *Visualization of Redundant Tokens Judgment:* As illustrated in Fig. 7, Our approach efficiently identifies and selects a substantial number of blank background patches in the original screenshot while minimally disrupting regions containing textual or iconographic elements. As evidenced, all

text and icons in the figure are fully preserved, and repetitive redundant parts are precisely marked. This selective process underscores the model's capability to distinguish between high- and low-information areas, thereby ensuring that redundant or irrelevant regions, which contribute little to the overall context, are reduced in focus. The visualization demonstrates the robustness of the method in preserving the integrity of essential visual components, providing the model with fine-grained features that contain richer visual information, thereby enhancing the recognition and localization capabilities for components in the screenshot.

V. DISCUSSION

A. Discussion on Textual Modality Enhancement

In the TME module, we did not incorporate all potential action types into the output space of the assistant. For example, in the AITZ dataset, operations, such as "press back," "press home," and "press Enter," exist but were not included in the output domain. This decision was primarily based on two key considerations.

First, press-related actions constitute an insignificantly small proportion in the AITZ dataset. If these were incorporated into

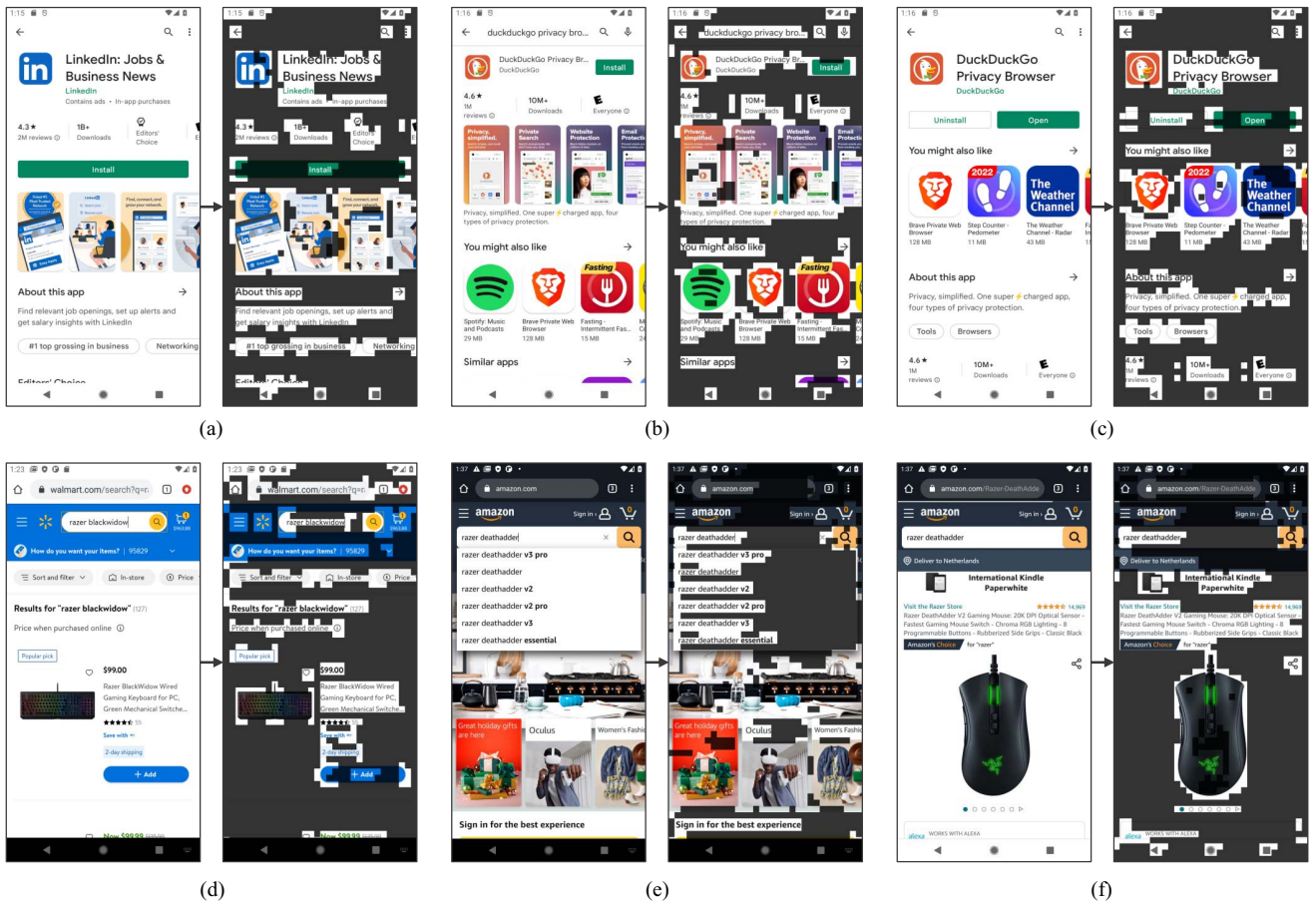


Fig. 7. Visualization results of redundant tokens judgment. (a)–(g) show different examples: left, the original screenshot; right, the visualization where blackened regions indicate patches judged as redundant.

the output space of the model, the number of potential outcomes would nearly double, thereby heightening the likelihood of erroneous judgments and consequently undermining the model's predictive accuracy. Moreover, expanding the action space with rarely used operations introduces unnecessary complexity without proportional benefits. Second, the assistant does not directly engage in the final decision-making process, so even if its output lacks specific operational details, subsequent models can accurately predict the appropriate actions based on contextual information. For instance, in scenarios where there is no historical input and the initial interface has minimal relevance to the task objective, the model can typically determine that the appropriate action is to “press home.” Therefore, restricting the output range of the assistant can effectively enhance its reliability with minimal impact on subsequent processes.

B. Error Cases Analysis

In GUI automation tasks, a single instruction may often correspond to multiple valid actions, and discrepancies in interpretation between human annotators and models are common. These factors can result in the model's predictions being marked as incorrect despite their objective validity. Fig. 8 presents several representative failure cases encountered during our experiments.

In the first example, the instruction is “What’s the news in Venezuela?” The query has already been entered into the search box, and MT-Agent predicts a click on the search button. However, the dataset label specifies pressing “Enter,” leading to a mismatch. Although both actions achieve the same goal, the prediction is judged incorrect. In the second case, the relevant news is already displayed, so MT-Agent infers that no action is needed. Yet, the dataset label requires scrolling down, possibly to retrieve more detailed information, again resulting in a judgment of failure. In the third case, the instruction is “How much does a 2-bedroom apartment rent for in Portland?” MT-Agent predicts a scroll-down action due to insufficient visible content, while the dataset indicates task completion, causing another mismatch.

Beyond label ambiguity, we also identify cases rooted in model misinterpretation. In the fourth example, with the instruction “delete browsing data in the chrome app,” MT-Agent incorrectly selects the upper-right cross icon. This suggests visual confusion, where the model erroneously associates the “X” icon with the notion of “clear” due to visual-semantic overlap. In the fifth example, the instruction is “Toggle notification dots,” the model correctly locates the relevant region but clicks on the label instead of the toggle switch, implying an incomplete understanding of the instruction's operational focus. Lastly, in the sixth case, with the instruction “Turn on JavaScript in the Chrome app,”

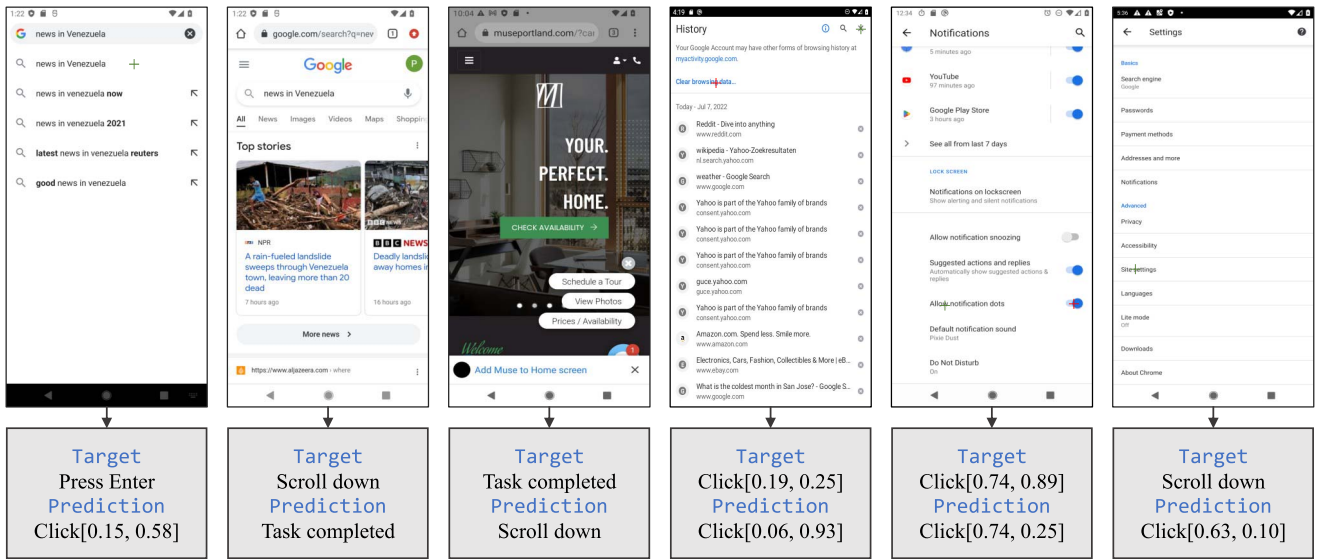


Fig. 8. Examples of inaccurate predictions made by the MT-Agent. “Target” refers to the label assigned to the corresponding screenshot in the dataset, while “prediction” represents the output generated by the MT-Agent. The green “cross” symbol represents the predicted click position when a click is predicted, while the red “cross” symbol indicates the labeled click position when a click is annotated.

MT-Agent chooses to click “Site settings” rather than scrolling down to reveal more options. While semantically reasonable, this still leads to incorrect task execution due to insufficient contextual awareness.

These failure patterns reveal that prediction errors can stem from both dataset annotation limitations and model-side challenges, including semantic misalignment and visual cue confusion, which underscore the need for more refined datasets with broader action equivalence definitions, as well as models with enhanced multimodal understanding and context reasoning capabilities.

C. Discussion on GUI Odyssey

The GUI Odyssey benchmark poses a unique challenge due to its emphasis on cross-app navigation, which requires stronger global reasoning and contextual understanding. As shown in Table IV, our MT-Agent performs slightly worse than the previous SOTA model, OdysseyAgent [13]. This gap can be largely attributed to two factors. First, OdysseyAgent uses a significantly larger LM, allowing it to better capture long-range dependencies and handle complex contextual information, which is useful in cross-app scenarios. Second, it incorporates historical screenshots into its input, providing richer context for decision-making. Effectively leveraging such temporal visual information requires substantial model capacity. Our lightweight MT-Agent, designed for efficiency, does not include the historical screenshots as input. This tradeoff partially limits MT-Agent’s performance on tasks requiring deep cross-state understanding.

D. Limitations and Future Work

While MT-Agent demonstrates strong performance on two diverse mobile GUI benchmarks, its architecture and design are specifically tailored for mobile application interfaces. This

domain features unique constraints, such as limited screen space, touch-based interaction patterns, and tightly coupled visual-linguistic cues. In contrast, Web-based GUIs or desktop applications often involve higher-resolution displays with more complex layouts, support for diverse input modalities (e.g., mouse hover and keyboard shortcuts), and multiwindow or dynamic content behaviors. These differences lead to a substantially larger and more heterogeneous action space, which our current architecture is not designed to handle, which fall outside the focus of this work. Additionally, as discussed in Section V-C, MT-Agent currently does not exploit historical visual context, which has been shown to be beneficial in complex cross-app scenarios like those in GUI Odyssey. Many recent approaches heavily depend on large-scale foundation models to tackle these problems, which contradicts our objective of designing a more lightweight and efficient model.

In future work, we plan to explore efficient strategies for integrating visual history and to investigate potential extensions of MT-Agent to support broader interface domains beyond mobile applications.

VI. CONCLUSION

In this article, we introduce MT-Agent, a GUI automation agent based on visual input. First, we present a modality enhancement module designed to enrich input features by Mining semantic information and providing fine-grained visual information, thereby extracting more sophisticated high-level representations and enhancing the model’s comprehension of GUI interfaces. Second, we propose a TDFF mechanism that leverages textual cues to facilitate the interaction between features from different modalities, effectively capturing and fusing multimodal information to improve both the understanding and planning of GUI tasks. Experimental results demonstrate that MT-Agent exhibits strong generalization and

robustness across various GUI tasks, thereby validating the effectiveness of our proposed approach.

REFERENCES

- [1] X. Liu et al., "AgentBench: Evaluating LLMs as agents," 2023, *arXiv:2308.03688*.
- [2] H. Furuta et al., "Multimodal Web navigation with instruction-finetuned foundation models," 2023, *arXiv:2305.11854*.
- [3] I. Gur et al., "A real-world Webagent with planning, long context understanding, and program synthesis," 2023, *arXiv:2307.12856*.
- [4] X. Deng et al., "Mind2Web: Towards a generalist agent for the Web," 2023, *arXiv:2306.06070*.
- [5] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, "GPT-4V(ision) is a generalist Web agent, if grounded," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 1–36. [Online]. Available: <https://openreview.net/forum?id=pieckJ2DiB>
- [6] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V," 2023, *arXiv:2310.11441*.
- [7] S. Sunkara et al., "Towards better semantic understanding of mobile interfaces," 2022, *arXiv:2210.02663*.
- [8] K. Cheng et al., "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguist. (Volume 1: Long Papers)*, Aug. 2024, pp. 9313–9332.
- [9] B. Gou et al., "Navigating the digital world as humans do: Universal visual grounding for GUI agents," 2024, *arXiv:2410.05243*.
- [10] K. You et al., "Ferret-UI: Grounded mobile UI understanding with multimodal LLMs," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 254–255.
- [11] Q. Wu et al., "MobileVLM: A vision-language model for better intra-and inter-UI understanding," 2024, *arXiv:2409.14818*.
- [12] W. Chen et al., "GUICourse: From general vision language models to versatile GUI agents," 2024. [Online]. Available: <https://x.com/arankomatsuzaki/status/1802908174690705629?lang=en>
- [13] Q. Lu et al., "GUI odyssey: A comprehensive dataset for cross-App GUI navigation on mobile devices," 2024, *arXiv:2406.08451*.
- [14] K. Zhao et al., "GUI testing arena: A unified benchmark for advancing autonomous GUI testing agent," 2024, *arXiv:2412.18426*.
- [15] S. Raimondo, C. Pal, X. Liu, D. Vazquez, and H. Palacios, "Improving generalization in task-oriented dialogues with workflows and action plans," 2023, *arXiv:2306.01729*.
- [16] D. Nguyen et al., "GUI agents: A survey," 2024, *arXiv:2412.13501*.
- [17] X. Ma, Z. Zhang, and H. Zhao, "CoCo-agent: A comprehensive cognitive MLLM agent for smartphone GUI automation," in *Findings of the Association for Computational Linguistics*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Assoc. Comput. Linguist., Aug. 2024, pp. 9097–9110. [Online]. Available: <https://aclanthology.org/2024.findings-acl.539/>
- [18] Z. Zhan and A. Zhang, "You only look at screens: Multimodal chain-of-action agents," 2023, *arXiv:2309.11436*.
- [19] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
- [20] R. Yang et al., "GPT4Tools: Teaching large language model to use tools via self-instruction," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–21.
- [21] D. Gao et al., "AssistGPT: A general multi-modal assistant that can plan, execute, inspect, and learn," 2023, *arXiv:2306.08640*.
- [22] C. Rawles, A. Li, D. Rodriguez, O. Riva, and T. P. Lillicrap, "Android in the wild: A large-scale dataset for android device control," 2023, *arXiv:2307.10088*.
- [23] R. Niu et al., "ScreenAgent: A vision language model-driven computer control agent," 2024, *arXiv:2402.07945*.
- [24] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [25] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 22199–22213.
- [26] Z. Zhang, A. Zhang, M. Li, and A. Smola, "Automatic chain of thought prompting in large language models," 2022, *arXiv:2210.03493*.
- [27] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [28] K. Lee et al., "Pix2Struct: Screenshot parsing as pretraining for visual language understanding," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 18893–18912.
- [29] A. Yan et al., "GPT-4V in wonderland: Large multimodal models for zero-shot smartphone GUI navigation," 2023, *arXiv:2311.07562*.
- [30] J. Hoscilowicz, B. Maj, B. Kozakiewicz, O. Tymoshchuk, and A. Janicki, "ClickAgent: Enhancing UI location capabilities of autonomous agents," 2024, *arXiv:2410.11872*.
- [31] J. Wang et al., "Mobile-agent: Autonomous multi-modal mobile device agent with visual perception," 2024, *arXiv:2401.16158*.
- [32] W. Hong et al., "CogAgent: A visual language model for GUI agents," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 14281–14290.
- [33] G. Li and Y. Li, "Spotlight: Mobile UI understanding using vision-language models with a focus," 2022, *arXiv:2209.14927*.
- [34] Z. Li et al., "Ferret-UI 2: Mastering universal user interface understanding across platforms," 2024, *arXiv:2410.18967*.
- [35] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [36] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 38–55.
- [37] J.-B. Alayrac et al., "Flamingo: A visual language model for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 23716–23736.
- [38] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," 2023, *arXiv:2304.08485*.
- [39] J. Li, D. Li, S. Savarese, and S. C. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, *arXiv:2301.12597*.
- [40] W. Wang et al., "CogVLM: Visual expert for pretrained language models," 2023, *arXiv:2311.03079*.
- [41] J. Wang et al., "Mobile-agent-v2: Mobile device operation assistant with effective navigation via multi-agent collaboration," 2024, *arXiv:2406.01014*.
- [42] J. Zhang et al., "Android in the zoo: Chain-of-action-thought for GUI agents," in *Proc. Find. Assoc. Comput. Linguist.*, Nov. 2024, pp. 12016–12031.
- [43] Q. Sun, Y. Fang, L. Wu, X. Wang, and Y. Cao, "EVA-CLIP: Improved training techniques for CLIP at scale," 2023, *arXiv:2303.15389*.
- [44] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 9992–10002.
- [45] G. Baechler et al., "ScreenAI: A vision-language model for UI and infographics understanding," 2024, *arXiv:2402.04615*.
- [46] "Flan-Alpaca: A fine-tuned language model combining flan-T5 and alpaca," 2024. [Online]. Available: <https://github.com/declare-lab/flan-alpaca>
- [47] Z. Wu et al., "OS-ATLAS: A foundation action model for generalist gui agents," 2024, *arXiv:2410.23218*.
- [48] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.
- [49] P. Wang et al., "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," 2024, *arXiv:2409.12191*.